

Contents

Qubes Secure SDLC / DevSecOps Lab — Final Technical Report	1
1. Executive Summary	2
2. Objective and Success Criteria	2
3. Qubes Architecture and Isolation Model	2
4. Repository and Evidence Layout	3
5. Vulnerable Baseline	3
6. Security Tooling	3
7. Before/After Results	4
8. Policy Gate	5
9. Remediation	5
10. Evidence Map	5
11. Why Separate VMs Matter	6
12. Limitations	6
13. Publication Readiness	6
14. Interview Defense Story	7
15. Final Conclusion	7
16. Control Mapping	7
17. Tool-by-tool Interpretation	8
Simgrep	8
Bandit	8
pip-audit	8
Gitleaks	8
Checkov	8
Trivy	8
Syft and Grype	8
18. Evidence Integrity Model	8
19. Release Decision Logic	9
20. Reviewer Checklist	9
21. Interview Q&A	9
22. Future Improvements	10
23. Final Reviewer Statement	10
24. Detailed Phase Chronology	10
25. Risk Acceptance Record	12
26. Reviewer Reproduction Guide	12
27. What Was Improved in the Final Revision	13
28. Why This Case Is Strong	13
29. Expected GitHub Reviewer Impression	13

Qubes Secure SDLC / DevSecOps Lab — Final Technical Report

Final revision date: 2026-05-09 07:34:58 UTC

Status: publication-ready

Main result: the vulnerable baseline is reproducibly blocked by the policy gate, and

the remediated version reproducibly passes.

1. Executive Summary

This project demonstrates a full Secure SDLC / DevSecOps case study with a SOC-style evidence workflow. In Qubes OS, development and documentation were performed in dev-workbench, while security tooling and scans were executed in security-runner. Evidence was transferred manually through the Qubes copy workflow, making every boundary crossing explicit.

The objective was achieved: the project includes a vulnerable baseline, SAST/SCA/secrets/IaC/containers evidence, a failed policy gate, a remediation branch, fixed scans, and a passed policy gate.

The strength of the case is not only the code. The main strength is provability: every phase contains command outputs, scan reports, summaries, project trees, Git evidence, and publication-readiness controls.

2. Objective and Success Criteria

Main objective: demonstrate what a Secure SDLC pipeline can look like in an isolated Qubes environment with SOC-style evidence discipline.

Success criteria:

1. The vulnerable version contains intentional issues in application code, dependencies, secrets, Dockerfile, and IaC.
2. Security tooling runs in a separate AppVM instead of being mixed with the development VM.
3. All checks produce machine-readable reports.
4. The vulnerable baseline fails the policy gate.
5. Remediation removes gate-blocking risks.
6. The fixed version passes the policy gate.
7. The evidence pack contains project trees, checksums, scan reports, SBOMs, final reports, and a presentation.
8. The repository is ready for public publication without transient cache or backup directories.

3. Qubes Architecture and Isolation Model

The project uses practical role separation:

- dev-workbench — repository, documentation, Git, reports, and evidence inventory.
- security-runner — Semgrep, Bandit, pip-audit, Gitleaks, Checkov, Trivy, Syft, and Grype.
- TemplateVM — base system packages only. User-space security tools were installed inside the AppVM where needed.
- Evidence transfer — explicit Qubes copy workflow.

This demonstrates not only tool knowledge, but also security engineering thinking: isolation, least trust, controlled evidence movement, and reproducible release evidence.

4. Repository and Evidence Layout

The repository is structured as a case study:

- app/vulnerable-version — intentionally vulnerable application.
- app/fixed-version — remediated application.
- docker — vulnerable and fixed Dockerfiles.
- iac/vulnerable and iac/fixed — Terraform examples.
- scripts — control scripts and policy gate.
- evidence/command_outputs — command execution evidence.
- evidence/scan_reports — JSON reports from security tools.
- evidence/sbom — CycloneDX SBOMs generated by Syft.
- evidence/before_after — matrices and summaries.
- reports — RU/EN final reports.
- presentation — storyboard, slide source, and HTML deck.
- docs/ru and docs/en — phase documentation.

5. Vulnerable Baseline

The vulnerable baseline is a controlled learning target. It contains common DevSecOps issues:

- insecure application patterns;
- vulnerable dependencies;
- lab secrets;
- weak IaC security group design;
- insecure Dockerfile patterns;
- a failing policy gate.

The baseline is preserved as a separate phase and tag for before/after comparison.

6. Security Tooling

The project uses multiple security control classes:

- Semgrep — SAST and multi-language rules.
- Bandit — Python security scanning.
- pip-audit — Python dependency SCA.
- Gitleaks — secret detection.
- Checkov — IaC scanning.
- Trivy — filesystem dependency scan, IaC/config scan, and Dockerfile scan.
- Syft — SBOM generation.
- Grype — SBOM vulnerability scan.

This gives broad DevSecOps coverage across code, dependencies, secrets, IaC, container configuration, SBOM, and policy decisioning.

7. Before/After Results

Control	Vulnerable baseline	Fixed version	Interpretation
Policy gate	FAILED	PASSED	FAILED baseline -> PASSED fixed release
Semgrep findings	10	0	all Semgrep findings removed
Semgrep ERROR findings	5	0	blocking severity removed
Bandit findings	7	0	all Bandit findings removed
Bandit HIGH findings	2	0	no HIGH issues in fixed app
pip-audit vulnerabilities	24	4	residual non-blocking dependency findings documented
Gitleaks findings	2	0	lab secrets removed from fixed app
Checkov failed checks	4	1	public ingress removed; residual unattached SG check documented
Checkov public ingress	4	0	0.0.0.0/0 removed
Trivy fs vulnerabilities	17	4	dependency risk reduced
Trivy fs HIGH/CRITICAL	6	0	no gate-blocking high/critical risk
Grype SBOM matches	17	4	SBOM scan retained for transparency
Grype HIGH/CRITICAL	6	0	no high/critical SBOM blockers
Syft SBOM components	5	3	smaller dependency surface

The fixed version is not presented as “zero output from every scanner”. Mature security engineering distinguishes gate-blocking findings from residual non-blocking noise. Residual findings are preserved and explained instead of being hidden.

8. Policy Gate

The policy gate is the key engineering control. It converts scan reports into a release decision:

- vulnerable baseline: POLICY_GATE_STATUS=FAILED;
- remediated version: POLICY_GATE_STATUS=PASSED.

Gate-relevant conditions include blocking SAST findings, Bandit HIGH issues, secrets, public ingress, HIGH/CRITICAL dependency/container findings, and HIGH/CRITICAL SBOM matches. After remediation, all gate-blocking counters are zero.

9. Remediation

Remediation was implemented as an engineering phase:

- application code: safer input handling and removal of lab secrets;
- dependencies: reduced and updated dependency surface;
- IaC: public ingress removed;
- Dockerfile: hardening and non-root execution;
- evidence: remediation matrix and repeat scans.

10. Evidence Map

Phase	What it proves	Artifact	Evidence
00-03	Базовая подготовка	docs/ru/03_qubes_vm_evidence/README, docs/ru/04_template_appvm_preparation.md	evidence/command_outputs/DEVSECOPS_00-03
04	Уязвимый baseline	app/vulnerable-version, docker/vulnerable.Dockerfile, iac/vulnerable/main.tf	evidence/command_outputs/DEVSECOPS_04
05	Security tooling	docs/ru/06_security_tooling/README	evidence/command_outputs/DEVSECOPS_05
06	Vulnerable scans	Semgrep, Bandit, pip-audit, Gitleaks, Checkov, Trivy, Syft, Grype	evidence/command_outputs/DEVSECOPS_06
07	Failed policy gate	scripts/08_policy_gate.sh	evidence/command_outputs/DEVSECOPS_07
08	Remediation	app/fixed-version, docker/fixed.Dockerfile, iac/fixed/main.tf	evidence/before_after/DEVSECOPS_08
09	Fixed scans + passed gate	fixed scan reports and SBOM	evidence/command_outputs/DEVSECOPS_09
10	Evidence pack	EVIDENCE_INVENTORY.md, SHA256 manifest, project trees	evidence/DEVSECOPS_10_SHA256
11	Technical report	reports/devsecops_lab_report/README and EN version	evidence/command_outputs/DEVSECOPS_11

Phase	What it proves	Artifact	Evidence
12	Presentation	presentation/devsecops_evidence_defense_and_outputs/DEV	evidence/defense_and_outputs/DEV
13	Publication cleanup	README, release notes, publication check	evidence/command_outputs/DEV

11. Why Separate VMs Matter

Separate VMs remain valuable even when evidence is copied manually:

1. Tooling VM can access the internet for databases and rule updates while the workbench stays controlled.
2. Security tools and their caches do not pollute the development VM.
3. Evidence transfer becomes an explicit trust-boundary event.
4. A compromised tool process does not automatically compromise the whole workspace.
5. The model resembles production security workflows where build, scan, evidence, and release are separated.

Manual Qubes GUI transfers do not weaken the case. They reinforce the operational security model.

12. Limitations

This is a lab, not a production deployment. Limitations:

- no live GitHub Actions runner with protected branches and secrets;
- security tools were executed manually inside a Qubes AppVM;
- IaC examples are demonstrational;
- residual non-blocking findings are kept for transparency;
- SBOM and vulnerability results depend on tool databases at scan time;
- screenshots are supplementary evidence, not the only proof.

13. Publication Readiness

Publication cleanup included:

- transient `__pycache__` removal;
- backup directory removal;
- semantic screenshot names;
- refreshed report pack;
- rebuilt evidence inventory and checksum manifest;
- added RU/EN PDF and HTML reports;
- final publication review.

14. Interview Defense Story

1. I built a Secure SDLC lab in Qubes OS with separated development and security scanning zones.
2. I created an intentionally vulnerable baseline.
3. I ran SAST, SCA, secrets, IaC, Trivy, SBOM, and Gripe checks.
4. The policy gate blocked the vulnerable baseline.
5. I remediated the application, dependencies, Dockerfile, and IaC.
6. The fixed version passed the policy gate.
7. Every command, report, SBOM, tree, checksum, document, and presentation artifact is stored as evidence.

15. Final Conclusion

The project goal was achieved. The repository demonstrates a full cycle: design -> vulnerable baseline -> security scans -> failed gate -> remediation -> fixed scans -> passed gate -> evidence pack -> report -> presentation -> publication readiness.

This is a strong portfolio case for DevSecOps, Secure SDLC, AppSec, security automation, and Qubes-based security workflows.

16. Control Mapping

The project can be read as a set of security controls, not just a lab repository.

Control area	Project implementation	Evidence
Source security	Semgrep and Bandit for application code	evidence/scan_reports/semgrep, evi-
Dependency security	pip-audit, Trivy fs, Gripe over SBOM	dence/scan_reports/bandit evidence/scan_reports/pip-audit, evi-
Secret hygiene	Gitleaks and custom .gitleaks.toml	dence/scan_reports/trivy, evi-
IaC security	Checkov and Trivy IaC	dence/scan_reports/gripe evidence/scan_reports/gitleaks
Container hardening	Trivy Dockerfile scan	evidence/scan_reports/checkov, evi-
Software transparency	Syft CycloneDX SBOM	dence/scan_reports/trivy evidence/scan_reports/trivy
Release decision	scripts/08_policy_gate.sh	evidence/sbom Phase 7 and Phase 9 command outputs
Evidence integrity	SHA256 manifest	evidence/DEVSECOPS_10_SHA256SUMS.t

17. Tool-by-tool Interpretation

Semgrep

Semgrep provides early detection of insecure coding patterns. The baseline proves the vulnerable state. In the fixed version, Semgrep findings are 0 and ERROR findings are 0.

Bandit

Bandit adds a Python-specific security view. In the fixed version, Bandit findings are 0, demonstrating that remediation affected the application layer, not only infrastructure files.

pip-audit

pip-audit exposes dependency risk. The fixed version still has 4 total vulnerabilities, preserved as residual non-blocking evidence. This is realistic: mature projects separate release-blocking risk from backlog risk.

Gitleaks

Gitleaks confirms that the fixed application contains no secret findings. Secret hygiene is critical because leaked secrets are a common real-world incident path.

Checkov

Checkov returns 1 failed check on fixed IaC. The gate-relevant condition, public ingress 0.0.0.0/0, is removed: fixed public ingress occurrences are 0.

Trivy

Trivy is used for filesystem vulnerabilities, IaC/config scanning, and Dockerfile scanning. In the fixed version, HIGH/CRITICAL filesystem vulnerabilities are 0 and Dockerfile HIGH/CRITICAL misconfigurations are 0.

Syft and Gripe

Syft produces SBOMs and Gripe scans them. This demonstrates supply chain transparency: the project records component composition and vulnerability matches.

18. Evidence Integrity Model

The evidence model follows three principles.

First, every important command stores stdout/stderr in evidence/command_outputs. This preserves date, qube, target, return code, and counters.

Second, machine-readable reports are stored separately from human summaries. JSON reports can be recalculated with tools like `jq`, while Markdown/CSV summaries remain readable.

Third, the SHA256 manifest makes post-build file changes visible. It is not a replacement for signed releases, but it is strong enough for a publication-ready portfolio case.

19. Release Decision Logic

The policy gate intentionally does not mean “every scanner returns zero”. It means agreed blocking conditions are absent.

Blocking examples:

- Semgrep ERROR findings.
- Bandit HIGH findings.
- Gitleaks findings.
- Public ingress in fixed IaC.
- Trivy HIGH/CRITICAL filesystem vulnerabilities.
- Trivy Dockerfile HIGH/CRITICAL misconfigurations.
- Grype HIGH/CRITICAL SBOM matches.

Non-blocking examples:

- Residual dependency findings not treated as HIGH/CRITICAL by the configured gate.
- Checkov design findings that do not reintroduce public ingress.
- Tool-specific noise that should be triaged rather than hidden.

A poor gate blocks everything and gets bypassed. A useful gate blocks what the team agrees is unacceptable for release.

20. Reviewer Checklist

A reviewer can validate the project without trusting the author:

1. Open `PROJECT_TREE.txt`.
2. Open `app/vulnerable-version` and `app/fixed-version`.
3. Compare `docker/vulnerable.Dockerfile` and `docker/fixed.Dockerfile`.
4. Compare `iac/vulnerable/main.tf` and `iac/fixed/main.tf`.
5. Open Phase 6 scan reports.
6. Confirm `POLICY_GATE_STATUS=FAILED`.
7. Open the remediation matrix.
8. Open Phase 9 fixed scan reports.
9. Confirm `POLICY_GATE_STATUS=PASSED`.
10. Verify `evidence/DEVSECOPS_10_SHA256SUMS.txt`.

21. Interview Q&A

Why not use one VM?

One VM mixes development trust, security tooling cache, internet access, and evi-

dence generation. Qubes allows role separation.

Why do some fixed findings remain?

Because gate design separates blocking risk from residual backlog. Residual findings are preserved, not hidden.

Why so much evidence?

Because this is a defensible case. In real security work, provenance matters as much as the final result.

What is the strongest part of the project?

The chain: vulnerable baseline -> failed gate -> remediation -> fixed scans -> passed gate.

What would you improve next?

CI workflow, signed releases, branch protection, SARIF uploads, container image scanning, dependency lockfiles, and automated report generation.

22. Future Improvements

A future version could include:

- GitHub Actions policy gate workflow.
- SARIF output for security review.
- Dependency lockfiles and reproducible Python environments.
- Container image build and image scan.
- Cosign signing for release artifacts.
- SLSA-style provenance notes.
- Renovate/Dependabot simulation.
- Makefile for local phase execution.
- Automated PDF/HTML report generation.
- GitHub release artifacts.

23. Final Reviewer Statement

At final build time, the project contains all elements of a strong DevSecOps portfolio case: clear architecture, vulnerable baseline, real scanner outputs, policy decisioning, remediation, fixed evidence, reports, presentation, checksums, and publication cleanup.

Main result: FAILED for the baseline and PASSED for the fixed version are proven by artifacts, not just described in prose.

24. Detailed Phase Chronology

Phase	Name	What was done	Value type
00	Project skeleton	Repository directories, evidence folders, branch/tags prepared	foundation
01	Qubes workflow	dev-workbench and security-runner separation documented	architecture
02	Template/AppVM preparation	Tool installation strategy clarified	operations
03	Security tools	Versions and local tool cache captured	toolchain
04	Vulnerable baseline	Application, Dockerfile and IaC vulnerable examples created	baseline
05	Security tooling docs	RU/EN tool usage notes added	documentation
06	Vulnerable scans	All scanner reports generated for baseline	assessment
07	Failed gate	Baseline blocked by policy gate	release control
08	Remediation	Fixed application, Dockerfile and IaC created	engineering fix
09	Fixed scans	Repeat scans and passed policy gate captured	verification
10	Evidence pack	Inventory, project tree and SHA256 manifest created	auditability
11	Report	Bilingual technical report created	communication
12	Presentation	Storyboard, slide source and HTML presentation created	defense
13	Publication cleanup	Backup/cache cleanup, final publication review	release readiness

25. Risk Acceptance Record

Not every residual finding must block publication of a lab project. The residual position is recorded explicitly.

Residual item	Decision	Why acceptable	Next step
Residual pip-audit findings	Documented as non-blocking backlog	No HIGH/CRITICAL gate blocker in configured policy	Keep visible in evidence and track in future CI
Checkov CKV2_AWS_5	Accepted for lab Terraform example	No public ingress remains	Attach SG to real resource in production IaC
Manual evidence transfer	Accepted Qubes operational model	Explicit boundary crossing is safer than implicit shared state	Automate only with controlled qrexec policy in future
No production deployment	By design	This is a portfolio lab	Add deployment only in a future isolated sandbox

26. Reviewer Reproduction Guide

Minimal verification path without rerunning the full project:

1. Read README.md and PORTFOLIO_SUMMARY.md.
2. Open reports/devsecops_lab_report_en.pdf.
3. Check PROJECT_TREE.txt.
4. Check vulnerable evidence:
 - evidence/scan_reports/semgrep/DEVSECOPS_06_REPORT_semgrep_vulnerable.json;
 - evidence/scan_reports/bandit/DEVSECOPS_06_REPORT_bandit_vulnerable.json;
 - evidence/scan_reports/gitleaks/DEVSECOPS_06_REPORT_gitleaks_vulnerable.json;
 - evidence/scan_reports/trivy/DEVSECOPS_06_REPORT_trivy_fs_vulnerable.json.
5. Check failed gate:
 - evidence/command_outputs/DEVSECOPS_07_OUTPUT_policy_gate_failed.txt.
6. Check remediation:
 - evidence/before_after/DEVSECOPS_08_REMEDIATION_MATRIX.csv.
7. Check fixed evidence:
 - evidence/scan_reports/semgrep/DEVSECOPS_09_REPORT_semgrep_fixed.json;
 - evidence/scan_reports/bandit/DEVSECOPS_09_REPORT_bandit_fixed.json;
 - evidence/scan_reports/gitleaks/DEVSECOPS_09_REPORT_gitleaks_fixed.json;
 - evidence/scan_reports/trivy/DEVSECOPS_09_REPORT_trivy_fs_fixed.json.
8. Check passed gate:
 - evidence/command_outputs/DEVSECOPS_09_OUTPUT_policy_gate_passed.txt.
9. Check integrity:
 - evidence/EVIDENCE_INVENTORY.csv;
 - evidence/DEVSECOPS_10_SHA256SUMS.txt.

27. What Was Improved in the Final Revision

The final revision strengthens the intermediate version:

- landing README is portfolio-ready;
- RU/EN reports are now full technical reports;
- PDF and HTML report versions were added;
- final publication review was added;
- screenshots received semantic names;
- backup directories were removed;
- SHA256 manifest was rebuilt;
- evidence inventory was rebuilt;
- release checklist was updated;
- phase 13 publication readiness was documented.

28. Why This Case Is Strong

The case is strong for three reasons.

First, it demonstrates process rather than isolated screenshots. It has baseline, failure, remediation, and passing result.

Second, it combines multiple DevSecOps tooling classes: SAST, SCA, secrets, IaC, container, SBOM, and policy gate.

Third, it is implemented in Qubes, where separation of duties is part of the design, not just decoration.

29. Expected GitHub Reviewer Impression

Expected repository impression:

- the structure is understandable on first open;
- README explains the project story;
- reports provide executive and technical levels;
- evidence can be verified without trusting the author;
- presentation is ready for defense;
- the vulnerable version is clearly marked as a controlled baseline;
- the fixed version and policy gate show engineering closure.